

PDF → JSON Project

STEP ID + STATUS Extraction — Complete Step-by-Step Guide

Goal: strengthen the STEP extraction part of the project so genuine STEP IDs are recognized and the parser supports **Success**, **Failure**, **Suspend**, **Progress**. The final status-position rule must still be verified against the problematic college PDF when it is available.

1. Important project rules

- The correct file is **pdf_parse.py**.
- The correct function is **extract_procedure_info(text)**.
- Do not change **main.py** for this step.
- Do not change `extract_procedure_info()`.
- Do not change timestamped output, page counting, procedure separation, PDFInfo, or JSON generation.
- The four supported statuses are **Success**, **Failure**, **Suspend**, **Progress**.
- A status word inside the STEP description must not automatically be treated as the actual status.

2. Make a backup

Copy the current working file before editing:

```
pdf_parse.py → pdf_parse_backup.py
```

3. Open the correct file

In PyCharm, open:

```
pdf_parse.py
```

Do not modify `main.py` for this step.

4. Keep `extract_procedure_info()` unchanged

Leave your existing `extract_procedure_info()` exactly as it is.

5. Replace only `extract_status()`

Delete the old `extract_status()` function. Then add the helper function below immediately before the new `extract_status()`.

```
def save_step_status(step_id, step_lines, status_data):

    # Join all lines belonging to this STEP
    step_text = " ".join(step_lines)

    # Look for a valid status
    status_match = re.search(
        r"\b(Success|Failure|Suspend|Progress)\b",
        step_text,
        re.IGNORECASE
    )

    if status_match:

        status = status_match.group(1).lower()
```

```

        if status not in status_data:
            status_data[status] = []

        status_data[status].append(step_id)

def extract_status(text):

    status_data = {}

    lines = text.splitlines()

    current_step_id = None
    current_step_lines = []

    for line in lines:

        line = line.strip()

        # Ignore empty lines
        if not line:
            continue

        # Check for a genuine STEP ID
        step_match = re.match(r"^(\d+\.\d+)\b", line)

        if step_match:

            # Finish previous STEP
            if current_step_id is not None:
                save_step_status(
                    current_step_id,
                    current_step_lines,
                    status_data
                )

            # Store new STEP ID
            current_step_id = step_match.group(1)

            current_step_lines = []

            # Get remaining text after STEP ID
            remaining_text = line[step_match.end():].strip()

            if remaining_text:
                current_step_lines.append(remaining_text)

        else:

            # Add line to current STEP
            if current_step_id is not None:
                current_step_lines.append(line)

    # Finish final STEP
    if current_step_id is not None:
        save_step_status(
            current_step_id,
            current_step_lines,
            status_data
        )

    return status_data

```

6. What the file should look like

```
import re

def extract_procedure_info(text):

    # YOUR EXISTING CODE
    # DO NOT CHANGE THIS

def save_step_status(step_id, step_lines, status_data):

    # code from Step 5

def extract_status(text):

    # code from Step 5
```

7. What the new logic does

```
PDF text
|
v
Find a line that STARTS with a STEP ID
|
v
01.0001
|
v
Collect the lines belonging to that STEP
|
v
When the next STEP ID is found,
finish the previous STEP
```

The STEP ID is recognized only at the beginning of a stripped line. This is important because a number such as 44.356 inside a time value should not automatically become a STEP ID.

8. Four supported statuses

```
Success
Failure
Suspend
Progress
```

They are stored as lowercase keys: success, failure, suspend, progress.

9. Expected JSON example

```
{
  "Procedure": "abbnm",
  "Time": "31-dec-2025 11:44:13.103",
  "Dur": "203.96(s)",
  "status": {
    "success": ["01.0001"],
    "failure": ["01.0002"],
    "suspend": ["01.0003"],
    "progress": ["01.0004"]
  }
}
```

10. Critical limitation

This code is a structural starting point. It supports all four status words, but it still searches the collected STEP text for the first matching status word. Therefore it is NOT yet the final protection against status words inside the STEP description.

```
01.0001
Check Success condition
11:44:20
Failure
```

The required result here is failure → 01.0001. The intermediate code could see Success first. Therefore, do not consider this extraction part finished until the actual college PDF layout has been checked.

11. Final structure we need

```
STEP ID
↓
STEP
↓
TIME
↓
STATUS
```

The final implementation must identify the actual STATUS position after the TIME field, rather than searching the entire STEP description.

12. Why the college PDF matters

PyMuPDF can extract the same visual table in different text layouts. Examples already seen in this project include STEP ID on one line, STEP/TIME/STATUS split across lines, and TIME + STATUS on the same line. The final parser must be based on the real extraction pattern.

```
Example A:
01.0001
some step description
11:44:20
Success
```

```
Example B:
01.0001 some step description
11:44:20 Success
```

```
Example C:
01.0001 some step description
11:44:20
Success
```

13. When you get the college PDF

1. Use one problematic PDF first; do not immediately process every PDF.
2. Inspect the PyMuPDF-extracted text around the wrong result.
3. Check the exact positions of STEP ID, STEP, TIME, and STATUS.
4. Test STEP descriptions containing Success, Failure, Suspend, and Progress.
5. Test time values containing decimal numbers such as 44.356.
6. Only after these cases work correctly should this extraction part be considered finished.
7. Then test several PDFs before starting the large next requirement.

14. Do not change these working parts

```
main.py
timestamped output filename
old output deletion
PDF page counting
PDFInfo
procedure separation
extract_procedure_info()
JSON generation
```

15. Final checklist

- pdf_parse.py is the file being edited.
- extract_procedure_info() is unchanged.
- main.py is unchanged.
- Success, Failure, Suspend, Progress are supported.
- STEP IDs are recognized from the beginning of a line.
- Time fragments are not deliberately treated as STEP IDs.
- A STEP description containing a status word cannot determine the final status in the finished implementation.
- The final status-position logic is verified using the real college PDF before moving to the next major step.

Bottom line: This guide gives you the exact file changes and the structured starting code. The final status-selection rule still needs the actual college PDF extraction to be verified; we should not pretend an unverified regex is universal.